Design Patterns - Lab 5-6
Persistence with Spring Data JPA

Goal

Learn how to persist your Java objects with Spring Data JPA to a relational database (JPA stands for Jakarta Persistence, formerly Java Persistence API).

Tasks

1. Java Embedded Databases - database management is embedded in your application. Overview at https://dzone.com/articles/3-java-embedded-databases
   Advantages: lightweight, fast, no external dependency, easy to configure. These are not to be used in production environments, rather they prove helpful for end-to-end tests

2. We'll use H2 Database, the main reason behind this is the ability to browse the database from a Web interface served from Spring's embedded Tomcat container. In order to configure your project to use Spring Data JPA and H2 database, add the following two dependencies to your build.gradle file

```
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
implementation 'com.h2database:h2:2.1.214'
```

3. Next, configure the database connection: create the file **application.properties** under src/main/resources folder with the following content

```
spring.datasource.url=jdbc:h2:file:~/h2db/booksdb
spring.datasource.username=sa
spring.datasource.password=
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.database=H2
spring.jpa.show-sql=true
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.na
ming.PhysicalNamingStrategyStandardImpl
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

Remarks:

- `~/h2db/booksdb` - is the path to the file where the entire database is saved (in this case a relative path to user's home directory)
- `spring.h2.console.enabled=true` - enables the H2 GUI console, which will be accessible under http://localhost:8080/h2-console
- `spring.jpa.show-sql=true` - enables logging of the Hibernate's generated SQL commands
- `spring.jpa.hibernate.ddl-auto=update` - creates/updates the database schema (code-first approach)

For more details about working with H2 from Spring Boot, see this post.

4. Transform the book model into JPA-manageable entities using JPA annotations.

> Tip: Use one annotation at a time, start the application and **inspect the Console to understand the SQL's CREATE / ALTER commands** generated by Hibernate to materialize the schema based on your code; Take a look at the database structure in H2 console.

Because annotating the model may be challenging, use a step-by-step approach, one annotation at a time, inspect the database before moving forward. Otherwise, you may end-up with multiple errors, which are difficult to debug and fix.

5. Implementation hints:
   - @Entity - on all classes whose instances need to be persisted to the database
     - Make sure that you add a default ctor for these classes (can use Lombok's @NoArgsConstructor(force = true) annotation to force generate one)
   - @Id - the primary key
     - Make sure that each Entity has an @Id attribute
     - Use @GeneratedValue to delegate the responsibility to generate unique ids to the database (otherwise, the application *needs* to provide unique ids for each object prior to persist it)
   - @Transient - use this annotation to avoid persisting fields/attributes you don't want to persist in the database, e.g., `ProxyImage.realImage`

- @Inheritance - use this annotation on the top-most class of your inheritance hierarchy
    - This needs to be a concrete class, not an interface (create one if you don't have one, e.g. BaseElement)
    - As the derived classes (book's elements) share little not no fields, use strategy = InheritanceType.JOINED
      Tip: in order to keep things simple in the database schema, you may start with InheritanceType.SINGLE_TABLE
- @OneToMany - use this annotation to persist the composition-like relationships in your model, such as Section □_ Element

```
@OneToMany(targetEntity = BaseElement.class)
private final List<Element> children = new ArrayList<>();
```

- @ManyToMany - use this annotation to persist the aggregation-like relationships in your model, such as Book □_ □Author

```
@ManyToMany
private List<Author> authors;
```

6. We're good now to hook up our Spring Data JPA's repositories into our project; create a subpackage named `persistence` and add there two interfaces
    - One repository for authors
    - One repository for books (this one will take care of the entire Element hierarchy!)

   For example, below is the books repository

```
@Repository
public interface BooksRepository extends JpaRepository<Book, Integer>  {
}
```

   This is all you need to save/read your Java objects to the relational database! Spring Data is magic :)

7. Update the presentation (Controller) and the service layers to make use of JPA repositories
    - Inject the BooksRepository into the BooksController and from here into the CommandContext
    - Update the Command's context to store the repository
    - Update CreateBookCommand implementation to do the following:

        - create a new Book instance (book)
        - Use the repository `book = booksRepository.save(book);` to save the book to the database
        - Return the created book to controller; you may want to return the book's Id in POST's response body
    - Update the other endpoints, getAll / getById / delete / update using a similar approach

> For each save/read operation, **inspect the Console to understand the SQL's SELECT / INSERT commands!**

8. The JSON representation of the object returned by ResponseEntity is automatically generated by Jackson library
    - It uses getXxx accessor methods to retrieve the fields that will be included in the JSON representation => make sure all the fields that you need in the JSON representation have a getter method like getFieldName
    - Add `@JsonIgnore` on getter methods if you want to exclude fields from JSON representation; for example, persisting to JSON a parent relationship may cause infinite recursion, hence you may want to exclude the parent from JSON serialization
    - Use `@JsonProperty("newPropName")` to change the field name in the JSON representation if you want a different name comparing to field name in Java

> Tip: Use an Internet browser extension to pretty format the JSON response received from your endpoints, such as JSON Formatter

9. Now that we have Spring Data JPA's setup and working, it is time to add an abstraction layer - the Data Access (Persistence) layer - sitting between the Controllers layer and JPA's repositories:
    - Add a CrudRespository interface abstracting the operation we need in upper layers (see Lecture 11-12 for details)
    - Add an Adapter adapting the Spring Data JPA repositories to our CrudRepository interface
    - Inject the CrudRespository instead of Spring Data repositories in BooksController and CommandContext instances.

Appendix: Troubleshooting frequently encountered errors

org.hibernate.TransientObjectException: object references an unsaved transient instance - save the transient instance before flushing: ro.uvt.models.Xxxx - by default, JPA/Hibernate persists only the object given to Repository.save(obj) method and not its linked/nested objects; in order to persist the entire aggregate, one needs to tell JPA to do so by including cascade=CascadeType.ALL to the @OneToMany @ManyToMany annotations that cause the exception to be thrown

javax.persistence.PersistenceException: Unable to build Hibernate SessionFactory; nested exception is org.hibernate.MappingException: Could not determine type for: Xxxx, at table: Yyyyy, for columns: [org.hibernate.mapping.Column(Zzzz)] - this means that the field Zzzz of class Yyyy is not of a basic type and it is not mapped to target class Xxxx; a @ManyToOne, @OneToOne annotation is usually missing

com.fasterxml.jackson.databind.exc.InvalidDefinitionException: No serializer found for class org.hibernate.proxy.pojo.bytebuddy.ByteBuddyInterceptor and no properties discovered to create BeanSerializer (to avoid exception, disable SerializationFeature.FAIL_ON_EMPTY_BEANS) (through reference chain:ro.uvt.models.Book$HibernateProxy$bJobNFaq["hibernateLazyInitializer"]) - this is a Jackson exception thrown when a Java object's JSON representation is built. When Hibernate fetches an object from the database, it creates a Proxy (hibernate.proxy) wrapping the *target* instance in order to support lazy-loading (remember our ImageProxy?). This proxy instance has additional fields,such as *hibernateLazyInitializer*, which Jackson tries to serialize to JSON representation, something that we obviously do not want. In order to avoid this, few options are available:

1) Disable serialization failures by adding the following line to your application.properties file
   ```
   spring.jackson.serialization.FAIL_ON_EMPTY_BEANS=false
   ```
   Important: Because this has a global effect and may hinder errors that we are interested in, *we shall not use this approach*!
2) Tell Jackson to ignore some fields in the proxy object when this gets serialized, and this is what we actually want; In order to do this, add the following annotation on the Entity that causes the issue (e.g. Book)
   ```
   @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
   ```